Drones, Deadlines and NP-Hardness

Metaheuristics for a Capacitated Medical Delivery Network

Team: [Member 1 · Member 2 · Member 3 · Member 4]

Instructor: Dr. Günet Eroğlu · Submitted 13 December 2026 · Code: github.com/batuhankiskac/AlgorithmProject

Abstract. We model the morning planning problem of a drone delivery start-up as the Capacitated Vehicle Routing Problem. We prove its decision version NP-complete through the chain Hamiltonian Cycle $\underset{p}{\leq}$ TSP $\underset{p}{\leq}$ CVRP, and show that deciding whether the customers fit into K vehicles is a strongly NP-complete Bin Packing problem. We also prove that the Bosphorus-crossing penalty preserves the triangle inequality. We implement, from scratch, a Clarke–Wright baseline, a memetic Genetic Algorithm with giant-tour chromosomes and an exact Split decoder, and a granular Tabu Search with $O(1)$ delta evaluation and adaptive capacity penalties. We derive the asymptotic cost of one generation and one iteration and confirm it on a log–log plot. Under equal time budgets and ten seeds, both metaheuristics reach the best-known solutions of A-n32-k5, A-n45-k6 and A-n60-k9 in every run. On A-n80-k10 the GA has a mean gap of 0.02% and TS 0.25%. Both reach the exact optimum of all tiny instances. The GA is significantly better on the larger instances (Wilcoxon $p < 0.001$).

1 Introduction

MediDrone Istanbul flies blood samples, vaccines and urgent medication from one central laboratory to clinics on both sides of the city. Every morning the operations team has to answer the same question: which drone visits which clinics, and in which order? Each clinic orders a different amount, a drone can carry at most Q units, every drone must come back to the lab, and crossing the Bosphorus costs extra because of the air-traffic corridor, wind and battery use. The CTO wants a good plan within seconds, not a proof of optimality, but also hard evidence that the method works and scales.

Stripped of the story, this is the **Capacitated Vehicle Routing Problem** (CVRP) [1]. It contains the Travelling Salesperson Problem (TSP) as the special case of one vehicle with unlimited capacity, and it contains Bin Packing as the question of whether the customers fit into K vehicles at all. Both are NP-hard, so we do not expect an exact polynomial-time algorithm. In Section 2 we make this precise with a chain of reductions, Hamiltonian Cycle $\underset{p}{\leq}$ TSP $\underset{p}{\leq}$ CVRP. We also show that the feasibility question alone is NP-complete and that the Bosphorus penalty keeps the distance a metric.

Our answer to the CTO is a toolbox of three algorithms, all implemented from scratch in Python (Section 3):

- **Clarke–Wright savings** [2]: a fast, deterministic constructive baseline, also used to start the other two methods.
- A **Genetic Algorithm** (GA) in the style of Prins [3]. Chromosomes are "giant tours" without route delimiters. An exact dynamic program (**Split**) cuts each one optimally into capacity-feasible routes.
- **Tabu Search** (TS) [4] works directly on routes. It uses four move types with $O(1)$ delta evaluation, a granular candidate list and an adaptive penalty for capacity violations.

Because this is an algorithms course, we do not stop at measuring. In Section 4 we derive the asymptotic cost of one GA generation and one TS iteration and check the predicted exponents on a log–log plot. Section 5 compares the methods under a fair protocol: equal wall-clock budgets, ten recorded seeds and gaps to the best-known solutions of the Augerat benchmarks [5], [6]. We also use tiny instances whose exact optimum we compute ourselves, and a team-specific MediDrone instance. Section 6 discusses what the data says, how the Bosphorus penalty shapes the routes, and the gap between proven approximation guarantees and what works in practice.

2 Problem Modelling and Complexity

In this chapter we pin down exactly which problem we are solving, show that its decision version is NP-complete, point out a second, less obvious NP-hard problem hidden inside it, and check that the "Bosphorus twist" of our MediDrone instance does not break any of the assumptions the theory relies on. We close with a short look at what is known about approximation algorithms for the CVRP.

2.1 Formal model

An instance of the Capacitated Vehicle Routing Problem (CVRP) is a tuple $I = (G, d, q, Q, K)$ where

- $G = (V, E)$ is a complete undirected graph with $V = \{0, 1, ..., n\}$; vertex 0 is the *depot* (for us: the MediDrone laboratory) and $V_c = \{1, ..., n\}$ are the *customers* (the clinics);
- $d : E \to \mathbb{R}^+$ assigns a distance to every edge and satisfies the triangle inequality $d(i, k) \leq d(i, j) + d(j, k)$ for all $i, j, k \in V$; we write $d(i, j) = d(j, i)$ since the graph is undirected;
- $q_i \in \mathbb{Z}^+$ is the demand of customer i, with $q_i \leq Q$;
- $Q \in \mathbb{Z}^+$ is the capacity of one vehicle (the drone payload) and $K \in \mathbb{Z}^+$ is the number of identical vehicles available.

A *route* is a sequence $R = (0, v_1, v_2, ..., v_m, 0)$ of distinct customers $v_1, ..., v_m$ that starts and ends at the depot. Its load and cost are

$$q(R) = \sum_{t=1}^{m} q_{v_t}, \quad c(R) = d(0, v_1) + \sum_{t=1}^{m-1} d(v_t, v_{t+1}) + d(v_m, 0). \tag{1}$$

A *feasible solution* is a set $S = \{R_1, ..., R_r\}$ of routes such that (i) $r \leq K$, (ii) every customer belongs to exactly one route, and it appears there exactly once, and (iii) $q(R_k) \leq Q$ for every k. Its cost is the sum of its route costs, $c(S) = \sum_{k=1}^{r} c(R_k)$.

- **Optimisation version (CVRP-OPT).** Given I, find a feasible solution S that minimises $c(S)$.
- **Decision version (CVRP-DEC).** Given I and a bound B, does a feasible solution S with $c(S) \leq B$ exist?

The two versions are closely linked: an algorithm for CVRP-OPT answers CVRP-DEC by comparing the optimum with B, so CVRP-OPT is at least as hard as CVRP-DEC. This is why it is enough to study the complexity of the decision version.

A remark on number representation: when distances are Euclidean, they are generally irrational square roots, and comparing a sum of square roots with B exactly is a famously delicate question. Like CVRPLIB [6] and most of the literature [1], we assume that the input distances are rational numbers (CVRPLIB rounds them to integers), given either as a matrix or computable in constant time from the input. All complexity statements below are made under this standard convention.

2.2 CVRP-DEC is in NP

Lemma 1 (membership). *CVRP-DEC $\in$ NP.*

Proof. **Certificate.** A certificate is the solution itself: a list of at most K routes, each written as its sequence of customer indices. Every customer appears in at most one place of an honest certificate, so the certificate contains at most n customer indices of $O(\log n)$ bits plus at most K route separators. Without loss of generality $K \leq n$ (more than n routes would need empty routes, which are useless), so the certificate has size $O(n \log n)$ bits, polynomial in the input size. If a claimed certificate is longer than this bound, the verifier rejects it immediately.

Verifier. Given I, B and the certificate, the verifier:

1. checks that the number of routes r is at most K;
2. creates a Boolean array `seen[1..n]`, initialised to false;
3. walks through every route; for each customer index v it checks $1 \leq v \leq n$ and `seen[v] = false`, then sets `seen[v] = true`, adds q_v to the route load and adds the distance from the previous vertex

(the depot at the start) to the running cost; at the end of the route it adds the distance back to the depot and checks that the route load is $\leq Q$;

4. checks that `seen[v] = true` for all v (every customer is served);
5. accepts if and only if all checks passed and the total cost is $\leq B$.

Steps 3 and 4 together guarantee that each customer is visited exactly once, so the verifier accepts exactly the feasible solutions with cost at most B.

Running time. Each customer index is handled once with $O(1)$ work (one array lookup, one demand addition, one distance lookup and addition), each route adds $O(1)$ work for its two depot edges and the load check, and step 4 is one pass over the array. The total is $O(n + K) = O(n)$ arithmetic operations on numbers whose bit length is polynomial in the input size, hence polynomial time. □

2.3 CVRP-DEC is NP-hard: the reduction chain

We show NP-hardness through the chain

$$\text{Hamiltonian Cycle} \quad \leq_p \quad \text{TSP-DEC (metric)} \quad \leq_p \quad \text{CVRP-DEC.} \tag{2}$$

Hamiltonian Cycle (HC: does an undirected graph H contain a cycle through every vertex exactly once?) is one of Karp's classical NP-complete problems [7], [8]. TSP-DEC asks, for a complete graph with edge weights w and a bound B, whether there is a tour (Hamiltonian cycle of the complete graph) of total weight at most B.

2.3.1 Step A: Hamiltonian Cycle $\leq_p$ metric TSP-DEC

> **Lemma 2**. *HC reduces in polynomial time to TSP-DEC restricted to instances whose weights satisfy the triangle inequality.*

Proof. **Construction.** Let $H = (U, F)$ be an HC instance with $m = |U| \geq 3$ vertices. Build the complete graph on U with weights

$$w(u, v) = \begin{cases} 1 \text{ if } \{u, v\} \in F \\ 2 \text{ otherwise,} \end{cases} \tag{3}$$

and set the bound $B = m$.

Metric. Every weight is 1 or 2, so for any three vertices $w(u, x) \leq 2 = 1 + 1 \leq w(u, v) + w(v, x)$. Hence w satisfies the triangle inequality, and it is symmetric and strictly positive. This matters for us: our CVRP definition *requires* a metric, so Step B can only start from a metric TSP instance. Reducing to general (non-metric) TSP would not be enough.

Correctness. A tour has exactly m edges, each of weight at least 1, so every tour weighs at least m, with equality if and only if all its edges have weight 1. ($\Rightarrow$) If H has a Hamiltonian cycle, the same cyclic order is a tour that uses only edges of F, so its weight is $m = B$. ($\Leftarrow$) If some tour has weight $\leq m$, its weight is exactly m, so all of its m edges have weight 1, i.e. they all lie in F. That tour is therefore a Hamiltonian cycle of H.

Running time. We write $\binom{m}{2} = O(m^2)$ weights, each found by one adjacency lookup, so the reduction runs in $O(m^2)$ time and its output has polynomial size. □

2.3.2 Step B: metric TSP-DEC $\leq_p$ CVRP-DEC

> **Lemma 3**. *Metric TSP-DEC reduces in polynomial time to CVRP-DEC.*

Proof. **Construction.** Let the TSP instance have cities $\{c_0, c_1, ..., c_n\}$ ($n \geq 2$), metric weights w and bound B. Build the CVRP instance with depot $0 := c_0$, customers $i := c_i$ for $i = 1, ..., n$, distances $d(i, j) = w(c_i, c_j)$, unit demands $q_i = 1$, capacity $Q = n$, a single vehicle $K = 1$, and the same bound B. Since w is a positive metric, d is a valid CVRP distance function, and $q_i = 1 \leq n = Q$.

Correctness. ($\Rightarrow$) Let $T = \left(c_0, c_{\pi(1)}, ..., c_{\pi(n)}, c_0\right)$ be a tour of weight $\leq B$ (any tour can be rotated so that it starts at c_0). Then $R = (0, \pi(1), ..., \pi(n), 0)$ is a single route visiting every customer exactly once,

with load $n \leq Q$; so $\{R\}$ is feasible with $K = 1$, and $c(R) = w(T) \leq B$. ($\Leftarrow$) Let S be a feasible CVRP solution with $c(S) \leq B$. Since $K = 1$ and there is at least one customer, S consists of exactly one route $R = (0, v_1, ..., v_n, 0)$, which by feasibility contains every customer exactly once. Thus $\left(c_0, c_{v_1}, ..., c_{v_n}, c_0\right)$ visits every city exactly once, it is a TSP tour, and its weight is $c(R) \leq B$.

Running time. We copy $O(n^2)$ distances and write n demands and three numbers, so the reduction takes $O(n^2)$ time. □

Note that the capacity constraint plays no role in this reduction (Q is large enough for everything); the hardness here comes entirely from the routing part. Section 2.4 shows that the capacity part is hard on its own.

2.3.3 Composing the reductions

Let f (Step A) run in time $O(p(N))$ and g (Step B) in time $O(r(N))$ for polynomials p and r, where N is the size of the respective input. A machine that runs in $p(N)$ steps cannot write more than $p(N)$ output symbols, so $|f(x)| \leq p(|x|)$. Running g on $f(x)$ therefore takes $O(r(p(|x|)))$ time, and the composition of two polynomials is again a polynomial. So $g \circ f$ is a polynomial-time reduction from HC to CVRP-DEC, and it is correct because each step preserves yes- and no-answers: $x \in$ HC $\Leftrightarrow f(x) \in$ TSP-DEC $\Leftrightarrow$ $g(f(x)) \in$ CVRP-DEC.

This is exactly how NP-hardness "travels" along the chain. Every problem $L \in$ NP reduces to HC in polynomial time, $L \leq_p$ HC, because HC is NP-complete. Composing that reduction with $g \circ f$ gives $L \leq_p$ CVRP-DEC for every $L \in$ NP, which is the definition of CVRP-DEC being NP-hard. If CVRP-DEC had a polynomial-time algorithm, we could solve every problem in NP in polynomial time, i.e. P = NP.

> **Theorem 1**. *CVRP-DEC is NP-complete, and CVRP-OPT is NP-hard.*

Proof. Membership is Lemma 1; NP-hardness follows from Lemmas 2 and 3 and the composition argument. For the optimisation version: a polynomial-time algorithm for CVRP-OPT would decide CVRP-DEC in polynomial time (compute the optimum c^* and answer "yes" iff $c^* \leq B$; if no feasible solution exists, answer "no"). □

Because the chain only produces instances with $K = 1$, the problem is hard even for one vehicle. It also stays hard for *planar Euclidean* distances, since Euclidean TSP is NP-hard as well [9]; so our geometric MediDrone setting does not hide an easy special case.

2.4 The hidden second NP-hard problem: fleet feasibility

The CVRP really combines two difficult problems. One is routing, which we have just seen. The other is *packing*: before we even talk about distances, we need to decide whether the customers can be split into at most K groups whose demands fit into the vehicles.

CVRP-FEAS. Given demands $q_1, ..., q_n \in \mathbb{Z}^+$, a capacity Q and a fleet size K: is there a partition of the customers into at most K groups $C_1, ..., C_r$ ($r \leq K$) with $\sum_{i \in C_k} q_i \leq Q$ for every k?

A CVRP instance has a feasible solution if and only if the answer to CVRP-FEAS is "yes". Indeed, a feasible solution gives such a partition (its routes), and conversely any valid partition becomes a feasible solution by visiting the customers of each group in an arbitrary order. In particular, CVRP-FEAS is the special case of CVRP-DEC where the bound B is so large (for example, the sum of all edge lengths) that the cost never matters.

2.4.1 Bin Packing $\leq_p$ CVRP-FEAS

Bin Packing (decision). Given item sizes $s_1, ..., s_n \in \mathbb{Z}^+$, a bin capacity C and a number k: can the items be packed into at most k bins of capacity C? This problem is NP-complete, and even *strongly* NP-complete [8]: it stays NP-complete when all numbers are bounded by a polynomial in n (equivalently, written in unary).

> **Theorem 2**. *CVRP-FEAS is NP-complete (in fact strongly NP-complete). Consequently, deciding whether a CVRP instance has* any *feasible solution is NP-complete.*

Proof. **Membership.** The certificate is an assignment $a : \{1, ..., n\} \to \{1, ..., K\}$ of customers to vehicles ($O(n \log K)$ bits). The verifier adds every q_i to the load of vehicle $a(i)$ and checks that no load exceeds Q, in $O(n + K)$ time.

Construction. Given a Bin Packing instance $(s_1, ..., s_n, C, k)$, we may assume $s_i \le C$ for all i (otherwise the answer is trivially "no" and we output a fixed "no" instance). Create one customer per item with $q_i = s_i$, set $Q = C$ and $K = k$, and use any valid metric, for example $d(i, j) = 1$ for all $i \neq j$ (which clearly satisfies the triangle inequality, as $1 \le 1 + 1$).

Correctness. Customers correspond one-to-one to items, vehicles to bins, and the load constraint $\sum_{i \in C_k} q_i \le Q$ is literally the bin constraint $\sum_{i \in C_k} s_i \le C$. So a packing into at most k bins is exactly a partition of the customers into at most K groups of load at most Q, and vice versa.

Running time. $O(n)$ for the numbers, plus $O(n^2)$ if the unit distance matrix is written out explicitly. The reduction does not change any number, so it maps instances with polynomially bounded numbers to instances with polynomially bounded numbers; this transfers *strong* NP-completeness. □

Strong NP-completeness has a practical meaning: unless P = NP, there is not even a *pseudo-polynomial* algorithm for CVRP-FEAS, i.e. one that is polynomial in n and in the numeric value of Q. So our small integer demands do not help in the worst case. The special case $K = 2$ is already hard: Partition (split integers with total $2S$ into two halves of sum S each) is the case $Q = S$, $K = 2$. Partition is only weakly NP-complete, and indeed for a fixed number of vehicles a subset-sum style dynamic program runs in pseudo-polynomial time, which is consistent with the above.

Consequences for approximation. Since even deciding whether a feasible solution with at most K vehicles exists is NP-complete, no polynomial-time algorithm can guarantee any finite approximation ratio for CVRP-OPT with a hard fleet limit unless P = NP: such an algorithm would in particular have to output a feasible solution whenever one exists, which would decide CVRP-FEAS. The approximation results in the last section therefore assume an unlimited fleet.

2.4.2 What this means for our metaheuristics

For a local search or a genetic algorithm, Theorem 2 says that even *staying feasible* is a nontrivial combinatorial task. How hard it is in practice depends on the *fill ratio*

$$\rho = \frac{\sum_{i=1}^{n} q_i}{KQ}, \tag{4}$$

the fraction of the total fleet capacity that the demand uses. When ρ is close to 1, there is almost no slack: only few partitions of the customers fit into K vehicles, and two such partitions can be far apart, with every path of small moves between them passing through overloaded solutions. The feasible region is then fragmented into many small "islands". Our two tight CVRPLIB benchmarks [5] are exactly of this type: A-n60-k9 has $\rho = 829/900 \approx 92\%$ and A-n80-k10 has $\rho = 942/1000 \approx 94\%$.

This is the theoretical reason behind two design decisions described in the algorithm chapters. Our Tabu Search [4] is allowed to visit *infeasible* intermediate solutions: it minimises $c(S) + \lambda \cdot \text{overload}(S)$, where $\text{overload}(S) = \sum_k \max(0, q(R_k) - Q)$ and the penalty weight λ is adapted during the search (raised while the search keeps being infeasible, lowered while it stays feasible), so that it can cross between islands instead of being trapped in one. Our genetic algorithm decodes giant tours with a *fleet-limited* Split procedure in the spirit of Prins [3], which only accepts segmentations that use at most K routes.

In our own MediDrone instance the situation is easier. The fleet is $K = \left\lceil \sum_i q_i / Q \right\rceil + 1$, so there is one full spare vehicle, i.e. at least Q units of unused capacity, and the demands ($q_i \le 30$) are small compared with $Q = 100$. Feasibility is still NP-hard in the worst case, but for such instances it is easy in practice: small items can be packed tightly, and the spare vehicle absorbs whatever packing loss remains.

2.5 The Bosphorus twist keeps the metric

Our MediDrone instance places all vertices in the square $[0, 1000]^2$. Let $\sigma(i) \in \{E, A\}$ be the side of vertex i (E, European, if $x_i < 500$, otherwise A, Asian; the depot at $(500, 500)$ is on the Asian side). With $e(i, j)$ the Euclidean distance and $P = 150$ the crossing penalty, the team distance is

$$d'(i,j) = e(i,j) + P \cdot [\sigma(i) \neq \sigma(j)], \tag{5}$$

where $[\cdot]$ is 1 if the condition holds and 0 otherwise. Since the CVRP definition (and the reductions above, the shortcutting arguments used by heuristics, and the approximation bounds below) all assume a metric, we have to check that the penalty does not destroy this property.

Lemma 4. *d' is symmetric, satisfies the triangle inequality, and $d'(i,j) > 0$ whenever i and j are at different locations.*

Proof. **Symmetry.** Both e and the condition $\sigma(i) \neq \sigma(j)$ are symmetric in i and j.

Positivity. $d'(i,j) \geq e(i,j) > 0$ for distinct locations. If two customers happened to share a location, d' would be 0 between them (they are then on the same side), so strictly speaking $d' : E \to \mathbb{R}^+$ needs distinct locations. With continuously distributed coordinates this essentially never happens, and even then d' is still a pseudometric, which is all that our algorithms and the arguments below use.

Triangle inequality. Fix i, j, k. We must show $d'(i,k) \leq d'(i,j) + d'(j,k)$. The Euclidean parts already satisfy $e(i,k) \leq e(i,j) + e(j,k)$, so it suffices to show that the penalty on the left is at most the total penalty on the right.

- *Case 1: $\sigma(i) = \sigma(k)$.* The left-hand side has no penalty, so $d'(i,k) = e(i,k) \leq e(i,j) + e(j,k) \leq d'(i,j) + d'(j,k)$ because penalties are non-negative.
- *Case 2: $\sigma(i) \neq \sigma(k)$.* The left-hand side is $e(i,k) + P$. Vertex j lies on one of the two sides, so it differs from exactly one of i and k: exactly one of the legs i–j and j–k crosses the Bosphorus. Hence the right-hand side equals $e(i,j) + e(j,k) + P \geq e(i,k) + P$.

In both cases the inequality holds. □

A one-line alternative. The indicator $\delta(i,j) = [\sigma(i) \neq \sigma(j)]$ is itself a pseudometric, the *cut metric* of the partition $\{E, A\}$: it is symmetric, $\delta(i,i) = 0$, and its triangle inequality is exactly Case 2 above. A non-negative combination of (pseudo)metrics is again a (pseudo)metric, because the triangle inequalities can simply be added. Therefore $d' = e + P \cdot \delta$ is a metric. The same argument would work for any number of regions and any non-negative crossing costs that themselves form a metric on the regions.

Lemma 4 means that the MediDrone instance is a valid CVRP instance in the sense of our formal model, so everything in this chapter applies to it. It also justifies a property our local search relies on: removing a customer from a route never increases that route's cost, because the shortcut $d'(u,w)$ is at most $d'(u,v) + d'(v,w)$.

2.6 Bonus: approximation algorithms for the CVRP

Since an exact polynomial algorithm is out of reach unless P = NP, one can ask for algorithms with a *guaranteed* ratio to the optimum. In this section we assume an unlimited fleet (see the remark after Theorem 2) and a metric d. Let OPT be the optimal CVRP cost and TSP* the optimal TSP tour length on V (depot included).

Two lower bounds. First, OPT $\geq$ TSP*: concatenating all routes of an optimal solution at the depot gives a closed walk that visits every vertex, and shortcutting repeated depot visits (allowed by the triangle inequality) turns it into a tour that is not longer. Second, the *radial bound*

$$\mathrm{OPT} \geq \frac{2}{Q} \sum_{i=1}^{n} q_i d(0,i). \tag{6}$$

To see this, take any route R of an optimal solution and its farthest customer v. By the triangle inequality, both the part of the route from the depot to v and the part back are at least $d(0,v)$ long, so $c(R) \geq 2 \max_{i \in R} d(0,i) \geq 2 \sum_{i \in R} \left(\frac{q_i}{Q}\right) d(0,i)$, where the last step uses $\sum_{i \in R} q_i \leq Q$ (a weighted average of the $d(0,i)$ with weights q_i/Q is at most their maximum). Summing over all routes gives the bound.

Iterated Tour Partitioning (ITP). Haimovich and Rinnooy Kan [10] proposed the following scheme: compute an α-approximate TSP tour through the depot and all customers, then cut it into consecutive segments of load at most Q, each served by a route from and back to the depot; the cutting point is chosen as the best among the Q possible offsets (or at random). The cost of the extra depot connections

can be charged against the radial bound, and the tour itself against $\text{TSP}^* \leq \text{OPT}$. For unit demands (equivalently, when demands may be split between vehicles) this yields a ratio of $1 + (1 - 1/Q)\alpha$. For unsplittable demands, Altinkemer and Gavish [11] showed a ratio of $2 + (1 - 2/Q)\alpha$.

With Christofides' algorithm [12], $\alpha = 3/2$, this gives about 2.5 for unit demands and about 3.5 for unsplittable demands. For many years these were the best known bounds. Blauth, Traub and Vygen [13] showed that both can be improved by a small but fixed constant (that is, the ratio becomes strictly below $\alpha + 1$ and $\alpha + 2$, respectively). On the TSP side, Karlin, Klein and Oveis Gharan [14] gave the first algorithm with a ratio slightly below 3/2 for metric TSP (the improvement is tiny, on the order of 10^{-36}), which can be plugged into the bounds above.

These guarantees are worst-case, and the constants are large compared with what metaheuristics achieve on real benchmarks (typically within a few percent of the best known solutions, see our experiments). Their value for us is conceptual: the radial bound and TSP^* give cheap lower bounds that tell us how far from optimal a solution can be at most, and the "giant tour, then cut" idea of ITP is exactly the principle behind the Split decoder in our genetic algorithm.

3 Methodology

All three methods share one representation: a solution is a list of routes, each a list of customers without the depot. Distances live in a precomputed $(n + 1) \times (n + 1)$ matrix, with CVRPLIB distances rounded with `nint`. We validated our parser by loading the published optimal solutions of all four benchmarks. They evaluate to exactly 784, 944, 1354 and 1763. An **independent feasibility checker** recomputes every reported solution from the raw coordinates and the Bosphorus rule, without reusing the solvers' distance matrix. It checks that every customer is served exactly once, that every load is at most Q, that there are at most K routes, and that the reported cost is correct.

3.1 Constructive baseline: Clarke–Wright savings

We use the classical **parallel** savings algorithm [2]. It starts from n routes $0 \to i \to 0$ and computes $s(i, j) = d(0, i) + d(0, j) - d(i, j)$ for all pairs. Merging a route that ends in i with a route that starts in j replaces the arcs $(i, 0)$ and $(0, j)$ by (i, j), which saves exactly $s(i, j)$. The pairs are scanned in decreasing order of saving. A merge is performed if i and j are endpoints of different routes and the combined load fits. A route is reversed when needed; this is free because d is symmetric.

Complexity. There are $n(n - 1)/2$ savings. Computing them takes $O(n^2)$ and sorting takes $O(n^2 \log n)$. The scan does $O(1)$ work per pair. At most $n - 1$ merges happen; each relabels the customers of one route and concatenates two lists in $O(n)$, so $O(n^2)$ in total. Overall $\boldsymbol{O(n^2 \log n)}$ **time and** $\boldsymbol{O(n^2)}$ **memory**, dominated by the sort. Clarke–Wright respects capacity but **ignores the fleet limit** $\boldsymbol{K}$. On A-n45-k6 it returns 7 routes for $K = 6$, a concrete instance of the hidden Bin Packing problem of Section 2.4. The metaheuristics therefore never trust the start solution to be K-feasible.

3.2 Genetic Algorithm with giant tours and Split

Following Prins [3], a chromosome is a **giant tour**: a permutation $T = (T_1, ..., T_n)$ of the customers without route delimiters. Every permutation is a valid chromosome, so standard permutation crossovers can be used without repair. The routes are recovered by an exact decoder.

Split as a shortest path. Build a DAG H with nodes $0, 1, ..., n$, where node i means "the first i customers of T are served". There is an arc (i, j) for $i < j$ iff $\sum_{k=i+1}^{j} q_{T_k} \leq Q$. Its cost is the cost of the trip $0 \to T_{i+1} \to ... \to T_j \to 0$. Each 0–$n$ path is a segmentation of T into feasible routes, and vice versa. Since arcs only go forward, Bellman's recurrence on the DAG solves it in one pass:

$$V(0) = 0, \quad V(j) = \min_{i<j,(i,j)\in A} \{V(i) + c(i, j)\}, \tag{7}$$

and the answer is $V(n)$, with routes recovered from the predecessor of each node.

Complexity. For a fixed i, the trip $(i, j + 1)$ is obtained from (i, j) in $O(1)$: $c(i, j + 1) = c(i, j) - d(T_j, 0) + d(T_j, T_{j+1}) + d(T_{j+1}, 0)$. The inner loop **stops at the first** $\boldsymbol{j}$ **whose load exceeds** $\boldsymbol{Q}$, because

demands are positive, so every longer trip is infeasible too. If a route holds at most L customers, each of the n outer iterations does at most L inner steps, giving $O(nL)$. In general $L \le n$, so $O(n^2)$. For our data $L \approx Q/\bar{q} \approx 6.5$, and Split is effectively linear. To respect the fleet limit we also implement the **layered** version $V_k(j) = \min_i\{V_{k-1}(i) + c(i,j)\}$ (exactly k routes), which costs $O(KnL)$.

Soft capacity. On tight instances almost no permutation has a capacity-feasible split into $\le K$ routes. We measured 0% of random tours on A-n45-k6 (fill 98.8%) and 22% on A-n80-k10. The GA therefore decodes with a soft layered Split: trips may carry up to $1.5Q$, and overload costs λ per unit. λ is adapted after every generation so that about 40% of the children are feasible (multiplied by 1.2 below 35%, by 0.85 above 45%). This follows Vidal et al.'s penalty idea [15]. The exact hard Split is applied again to every feasible local-search result.

Operators.

- **Initial population.** The Clarke–Wright giant tour, 30% **sweep** orders (customers sorted by polar angle around the depot), and random permutations. Clones are rejected.
- **Selection.** Binary tournament: $O(1)$, mild pressure, no fitness scaling needed.
- **Crossover.** Order Crossover (OX) with $p_{\text{cx}} = 0.9$. It copies a slice of parent 1 (consecutive customers, i.e. partial routes) and fills the rest in parent 2's relative order, in $O(n)$.
- **Mutation.** Segment inversion with $p_{\text{mut}} = 0.2$. It changes only two adjacencies of the tour (a 2-opt move), so it perturbs without destroying route structure.
- **Memetic local search** (extra credit). Applied to a child with probability $p_{\text{ls}} = 0.5$. It uses the same move types as the Tabu Search (relocate, swap, 2-opt, both 2-opt* variants) on a granular neighbourhood of the 20 nearest customers, with first improvement and $O(1)$ deltas. An overloaded result is repaired with probability 0.5 by re-running the search with 10λ and then 100λ.
- **Replacement and diversity.** $(\mu + \lambda)$ truncation with $\mu = \lambda_{\text{off}} = 25$, which is elitist by construction. A child whose fitness is within $\Delta = 0.5$ of an existing individual is rejected (Prins' spacing rule). After 60 generations without improvement, all but the 2 best individuals are replaced by new random ones.

3.3 Tabu Search

The Tabu Search works directly on routes. It always keeps exactly K route slots, some possibly empty, so the fleet limit can never be violated; capacity may be. The search starts from Clarke–Wright. If that uses more than K routes, the routes with the smallest load are dissolved and their customers reinserted at the cheapest position, accepting overload.

Objective and capacity handling. We minimise $f(S) = c(S) + \lambda \cdot \sum_r \max(0, L_r - Q)$, where L_r is the load of route r. We chose to **allow** infeasible moves. Section 2.4 shows that feasibility itself is NP-hard, and on 92–99% full instances a feasible-only search gets trapped between few feasible regions. λ adapts every iteration, as in Cordeau et al. [16]: it is multiplied by 1.5 if the current solution is overloaded and divided by 1.5 otherwise. It starts at $c(S_0)/\sum q$. The search thus oscillates around the feasibility border; about half of the iterations end feasible. The best **feasible** solution is returned.

Neighbourhood and candidate list. Four move families are used: relocate (inside a route or to another route, including an empty one), swap (between routes), 2-opt (inside a route) and 2-opt* (exchanging route tails, in both orientations). To keep iterations fast we use a **granular** candidate list [17]: for each customer u and each of its $k = 20$ nearest neighbours v, only moves that make u adjacent to v are evaluated. The best admissible move is applied even if it worsens f.

Delta evaluation. Every move changes a constant number of arcs, so its cost change is computed in $O(1)$. For relocating u (between p and s) to a position between a and b:

$$\Delta_{\text{reloc}} = \underbrace{d(a,u) + d(u,b) - d(a,b)}_{\text{insertion cost}} - \underbrace{(d(p,u) + d(u,s) - d(p,s))}_{\text{removal saving}}. \tag{8}$$

The other formulas are analogous. A 2-opt move reversing $R[i..j]$ costs $d\big(R_{i-1}, R_j\big) + d\big(R_i, R_{j+1}\big) - d(R_{i-1}, R_i) - d\big(R_j, R_{j+1}\big)$; the same holds for 2-opt* and swap. Load changes are also $O(1)$: we maintain route loads and per-customer prefix loads, so a 2-opt* tail exchange gives $L_1' = P(u) + L_2 - P(p_v)$. Unit tests compare every formula against full recomputation on thousands of random and edge-case moves.

Tabu list and aspiration. The attribute is an **arc**. When a move removes the undirected arc $\{a, b\}$, recreating it is tabu for $\theta \sim U[5, 15]$ iterations, drawn per arc. A move is tabu if any arc it creates is tabu. Arcs work uniformly for all move types; a "(customer, route)" attribute would be ambiguous after a 2-opt* tail exchange. Tabu status is stored in an $(n+1)^2$ matrix of expiry iterations, so checking is $O(1)$. **Aspiration**: a tabu move is allowed if it yields a feasible solution better than the best found so far.

Diversification and stopping. After $40n$ iterations without a new best, the search restarts from the best solution with a ruin-and-recreate step. It removes 10–25% of the customers and reinserts them at their cheapest penalised position. The search stops at the wall-clock budget.

```
SPLIT(T):                          // T =
giant tour
  V[0] = 0; V[1..n] = INF
  for i = 0 .. n-1:
    load = 0; j = i + 1
    while j <= n:
      load += q[T_j]
      if load > Q: break           // => O(n
L)
      cost = (j == i+1) ? d(0,T_j) +
d(T_j,0)
           : cost - d(T_{j-1},0)
             + d(T_{j-1},T_j) + d(T_j,0)
      if V[i] + cost < V[j]:
        V[j] = V[i] + cost; P[j] = i
      j += 1
  return V[n], routes read from P
```

Algorithm 1: Split (Prins) with capacity cut-off

```
CW(I):
  each customer i: route {i}
  S = [(d0i + d0j - dij, i, j) : i < j]
  sort S by decreasing saving   // O(n^2
log n)
  for (s, i, j) in S:
    A = route(i); B = route(j)
    if A == B or load(A)+load(B) > Q:
      continue
    if i, j not endpoints: continue
    orient A to end in i, B to start in j
    merge A + B
  return routes
```

Algorithm 2: Clarke–Wright savings

```
GA(I, T_max):
  P = decode(CW tour, sweep tours, random
tours)
  while time < T_max:                //
one generation
    C = {}
    repeat lambda_off times:
      p1, p2 = TOURNAMENT(P), TOURNAMENT(P)
      t = OX(p1, p2) w.p. p_cx else
copy(p1)
      INVERT random segment of t  w.p.
p_mut
      x = SOFT-SPLIT(t, K, lambda)
      w.p. p_ls: x = LOCAL-SEARCH(x); re-
split
      if x not within Delta of P u C: C +=
x
    adapt lambda to 40% feasible children
    P = mu best of P u C              //
elitist
    if 60 gens w/o improvement: restart
  return best feasible solution seen
```

Algorithm 3: Memetic GA

```
TS(I, T_max):
  S = CW(I); reduce to <= K routes
  lambda = c(S)/sum q; best = S if feasible
  while time < T_max:                //
one iteration
    m* = argmin over u, v in N_k(u), moves
         creating arc (u,v):  dc(m) +
lambda*do(m)
         s.t. not tabu(m) or aspiration(m)
    make arcs removed by m* tabu
       for U[5,15] iterations
    S = S + m*                         //
O(n)
    lambda *= 1.5 if overloaded else /= 1.5
    if feasible and c(S) < c(best): best =
S
    if 40n iterations w/o improvement:
      S = RUIN-RECREATE(best)
  return best
```

Algorithm 4: Tabu Search

4 Complexity Analysis of Our Own Code

We use n for the number of customers, K for the fleet size and L for the maximum number of customers on one route; for our data $L \approx Q/\bar{q} \approx 6.5$. The GA has population μ and λ_{off} children per generation. The local search and the Tabu Search use $g = k = 20$ granular neighbours.

4.1 One GA generation

One generation creates λ_{off} children and then performs replacement. Per child:

- two binary tournaments: $O(1)$
- OX crossover and inversion: $O(n)$ each
- soft layered Split: $O(KnL')$ with $L' \approx 1.5L$
- duplicate test: $O(\mu + \lambda_{\text{off}})$

Once per generation, the penalty update, $(\mu + \lambda_{\text{off}})$ truncation (a sort) and the diversity measurement (pairwise edge-set distance) cost $O((\mu + \lambda_{\text{off}}) \log(\mu + \lambda_{\text{off}}) + \mu^2 n)$. For the plain GA

$$T_{\text{gen}} = O(\lambda_{\text{off}}(KnL + \mu + \lambda_{\text{off}}) + \mu^2 n), \tag{9}$$

which is $O(\lambda_{\text{off}} K n^2)$ in the worst case $L = n$. In our setting $K \approx n/6.5$ and L is constant, so the Split term behaves like $\Theta(n^2)$ but with a small constant, and the diversity term is linear in n. The memetic GA adds, for a fraction p_{ls} of the children, a local search that costs $\Theta(ng)$ per pass over all customers, plus $O(L)$ per applied move. The number of passes is not bounded polynomially a priori, since each applied move only has to decrease f, but empirically it is a small constant. We therefore predict per-generation growth between n^1 (local-search dominated) and n^2 (Split dominated).

4.2 One TS iteration

One iteration scans every customer u and its k candidates. Each pair yields at most five moves, and each is evaluated in $O(1)$: an arc delta, a load update from prefix loads, and a tabu-matrix lookup. Applying the chosen move and rebuilding the position, predecessor, successor and prefix-load arrays of the two touched routes costs $O(n)$, and the tabu and λ updates cost $O(1)$. Hence

$$T_{\text{iter}} = \Theta(nk) + O(n) = \Theta(n) \quad \text{for fixed } k. \tag{10}$$

Without the candidate list, the full relocate/swap neighbourhood has $\Theta(n^2)$ moves, so an iteration would cost $\Theta(n^2)$ even with delta evaluation. Without delta evaluation each move costs $\Theta(n)$, giving $\Theta(n^3)$.

4.3 What delta evaluation saves

Re-evaluating a neighbour from scratch costs $\Theta(n)$: copying and changing the routes, then summing $n + K$ arcs. A delta costs $O(1)$. With about $5nk$ candidate moves per iteration, delta evaluation turns $\Theta(n^2 k)$ work into $\Theta(nk)$, a factor of $\Theta(n)$. We measured both methods on 5,000 random moves of all types (`moves.delta_vs_full_benchmark`). The function also asserts that the two methods agree.

n	delta (µs/move)	full re-evaluation (µs/move)	speed-up	recompute cost only
25	0.21	1.9	8.8×	1.3 µs
50	0.22	3.2	14.6×	2.5 µs
100	0.21	5.6	27.4×	4.5 µs
200	0.23	11.2	49.2×	9.3 µs
400	0.34	26.5	78.7×	21.9 µs
800	0.4	50.2	124.2×	44.8 µs

Table 1: Delta evaluation vs. full re-evaluation of a move. The delta cost stays flat while full evaluation grows linearly with n.

4.4 Empirical validation

We generated MediDrone-style instances with $n \in \{25, 50, 100, 200, 400, 800\}$ ($Q = 100$, demands $U[1, 30]$). For each we timed a solver run of a and of b iterations from the same Clarke–Wright start. We took $(t_b - t_a)/(b - a)$ as the time per iteration, which cancels the set-up cost, and used the median of three repetitions. Figure 1 shows the results on a log–log scale. The fitted slopes are:

- GA generation: **1.51**
- TS iteration: **1.04**
- Split: **1.05**
- Clarke–Wright: **2.2**

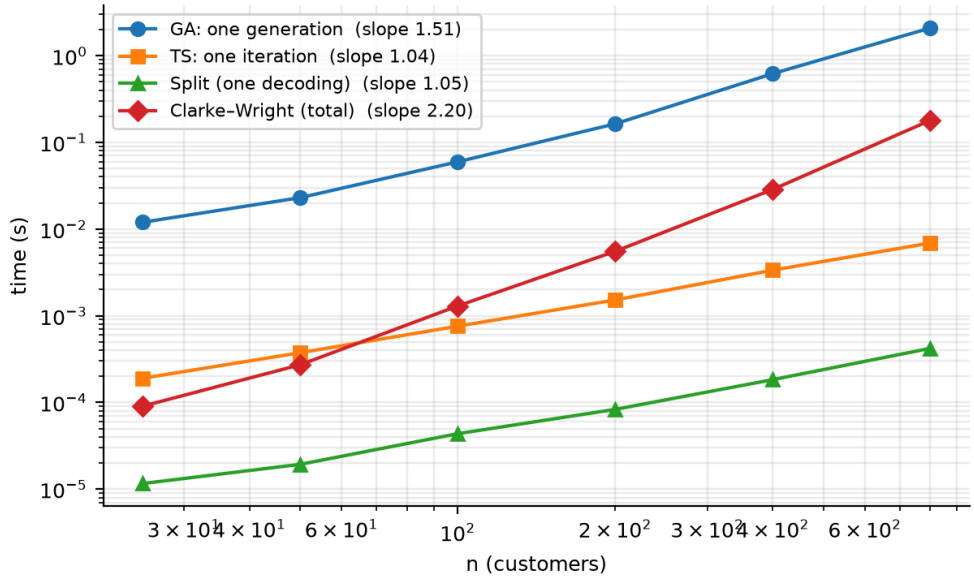


Figure 1: Measured time per unit of work vs. n (log–log) with least-squares slopes.

Interpretation. The measurements match the analysis.

- **TS iteration.** The slope is close to 1, as predicted by $\Theta(nk)$ with fixed k. Without the candidate list it would be 2.
- **Split.** The unlimited Split has slope about 1. This is the $O(nL)$ bound with $L \approx 6.5$ independent of n, not the worst case $O(n^2)$.
- **Clarke–Wright.** The slope is slightly above 2, consistent with $O(n^2 \log n)$: over $n \in [25, 800]$, the extra $\log n$ factor adds about 0.2 to the fitted slope.
- **GA generation.** Its slope lies between the two terms identified above. The fleet-limited Split costs $O(KnL)$ with $K \approx n/6.5$, so it grows quadratically. The local search, which dominates at small n, grows roughly linearly. As n grows the quadratic term takes over, and the local slope between $n = 400$ and 800 is about 1.75.

Table 1 shows the other half of the story. A delta evaluation costs a constant 0.2–0.4 µs, while a full re-evaluation grows linearly. The speed-up rises from about 15× at $n = 50$ to over 100× at $n = 800$, exactly the predicted factor $\Theta(n)$. In pure Python, this is the difference between about 1,300 and about 20 TS iterations per second at $n = 100$.

5 Experiments

5.1 Set-up

Hardware and software. Apple M5 (10 cores, of which 4 are performance cores) with 16 GB RAM, Python 3.14.10 with NumPy. All solvers are pure Python. Runs were executed 4 at a time, each in its own process. The machine was also used for other work during the experiments, so absolute iteration counts are lower than on an idle machine. GA and TS ran interleaved under the same conditions, so the comparison stays fair.

Protocol. Each stochastic algorithm ran 10 times per instance with the recorded seeds 11, 23, 37, 41, 53, 67, 79, 83, 97, 101. GA and TS received **the same wall-clock budget**: 2 s for the tiny instances, 30 s for $n \leq 50$ and 60 s for larger n. Both start from the Clarke–Wright solution, the GA as one individual of its initial population. We report the gap to the best-known solution (BKS) from CVRPLIB, Gap $= 100 \cdot (\text{Cost} - \text{BKS})/\text{BKS}$. For our MediDrone instance no BKS exists, so gaps there are relative to the best cost found by any run. Every solution passed the independent checker; the only exception is noted in Table 2. All parameters are listed in Section 3. Raw results for every run are in `results/runs.csv`.

5.2 Results on the benchmarks and the team instance

Instance	Alg.	Best	Mean	Std	BKS	Best gap %	Mean gap %	Time to best (s)	Iter.
A-n32-k5	CW	842	842	0	784	7.4	7.4	–	–
	GA	784	784	0	784	0	0	0	725
	TS	784	784	0	784	0	0	0.1	75873
A-n45-k6	CW	997	997	0	944	5.61	5.61	–	–
	GA	944	944	0	944	0	0	0.4	1047
	TS	944	944	0	944	0	0	0	62556
A-n60-k9	CW	1408	1408	0	1354	3.99	3.99	–	–
	GA	1354	1354	0	1354	0	0	3.6	720
	TS	1354	1354	0	1354	0	0	18.3	48635
A-n80-k10	CW	1840	1840	0	1763	4.37	4.37	–	–
	GA	1763	1763.4	0.97	1763	0	0.02	24.2	447
	TS	1763	1767.4	5.3	1763	0	0.25	26.4	26931
MediDrone	CW	15647.5	15647.5	0	–	–	–	–	–
	GA	14708.17	14708.17	0	–	–	–	21.6	283
	TS	14722.16	14743.46	13.5	–	–	–	36.6	28132

Table 2: Results over 10 runs, with a 30 s budget for $n \leq 50$ and 60 s otherwise. "Iter." is the mean number of GA generations or TS iterations. On A-n45-k6, Clarke–Wright uses 7 routes for $K = 6$, so it is not a feasible CVRP solution; its cost is listed for reference only.

Instance	Optimum (DP)	CW cost	CW gap %	GA hits optimum	TS hits optimum
tiny-n7	3884.47	4130.53	6.33	10 / 10	10 / 10
tiny-n8	3387.94	3387.94	0	10 / 10	10 / 10
tiny-n9	5835.41	5965.63	2.23	10 / 10	10 / 10

Table 3: Tiny instances with exact optima (Held–Karp + set-partition DP), 2 s per run.

Observations. Both metaheuristics are far ahead of the constructive baseline, which is 4–7% above the BKS. The memetic GA ends at the BKS in 38 of its 40 benchmark runs; the other two are A-n80-k10 runs that end at 1764 and 1766. The Tabu Search also reaches the BKS on A-n32-k5, A-n45-k6 and A-n60-k9 in all runs. On A-n80-k10 it reaches the BKS once in 10 runs (mean gap 0.25%). On the tiny instances both methods find the DP optimum in 10/10 runs (Table 3), so neither is missing optimal solutions because of a bug in the search space or the decoder. Clarke–Wright happens to be optimal on tiny-n8.

On our MediDrone instance, all 10 GA runs converge to the **same** plan (cost 14 708.17, standard deviation 0). TS runs end between 14 722 and 14 762, at most 0.4% above it, and Clarke–Wright is 6.4% above. Ten independent runs agreeing to the last decimal, combined with the GA's record on the benchmarks, suggests that this plan is optimal or very close to it. We cannot prove this, because the instance is far too large for our exact DP.

5.3 Convergence

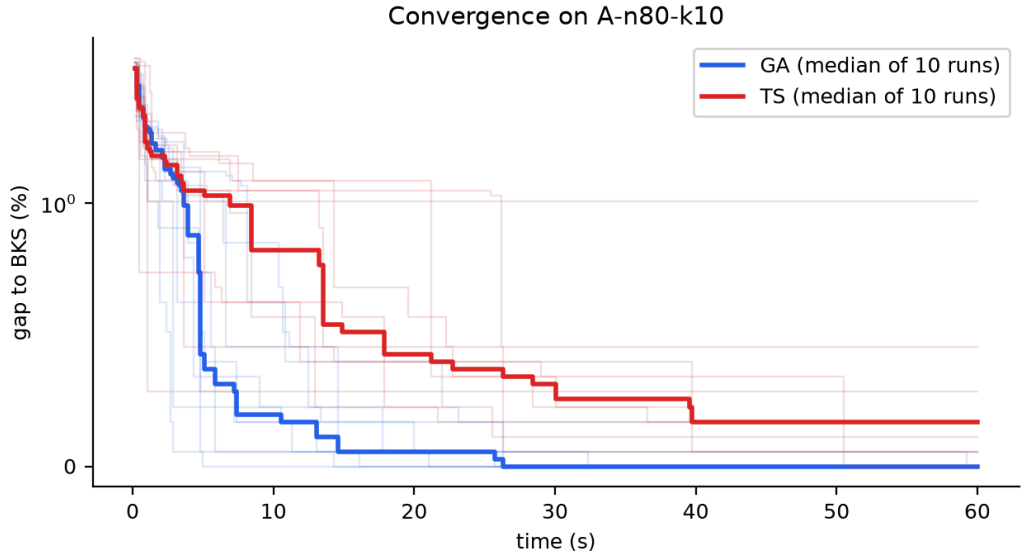


Figure 2: Best cost vs. time on A-n80-k10 (thin lines: individual runs; thick: median of 10). The symlog axis is linear below 1%.

Figure 2 shows a typical picture. The first 3 s look alike, because both methods start from Clarke–Wright (4.4%) and both quickly remove its obvious defects. After that the GA's median drops below 0.5% within 6 s and reaches the BKS after about 26 s. TS keeps improving in small steps and its median ends at 0.17%. TS performs about 60 times more iterations than the GA performs generations, but each iteration is a single move, whereas a GA generation runs 25 complete decodings and about 12 local searches. Convergence plots for the other instances are in `figures/`.

5.4 Statistical comparison (bonus)

Instance	Test	GA median	TS median	p-value
A-n32-k5	Mann-Whitney U (Wilcoxon rank-sum)	784	784	1
A-n45-k6	Mann-Whitney U (Wilcoxon rank-sum)	944	944	1
A-n60-k9	Mann-Whitney U (Wilcoxon rank-sum)	1354	1354	1
A-n80-k10	Mann-Whitney U (Wilcoxon rank-sum)	1763	1766	0.0025
MediDrone	Mann-Whitney U (Wilcoxon rank-sum)	14708.17	14744.47	0.000064
all (paired by instance, seed)	Wilcoxon signed-rank	0	0	0.000249

Table 4: Two-sided Mann–Whitney U test per instance (10 vs. 10 runs) and Wilcoxon signed-rank test [18] on the relative gaps of all benchmark and MediDrone runs, paired by (instance, seed). For the paired test the "median" columns show the median gap in %.

On the three smaller benchmarks the two methods are statistically indistinguishable, because both always find the BKS ($p = 1$). On A-n80-k10 ($p \approx 0.002$) and on MediDrone ($p < 10^{-4}$) the GA is significantly better. Across all 50 paired runs the GA wins 18, TS wins 1 and 31 are ties. The signed-rank test rejects equality ($p \approx 2.5 \cdot 10^{-4}$).

5.5 The MediDrone plans and the Bosphorus

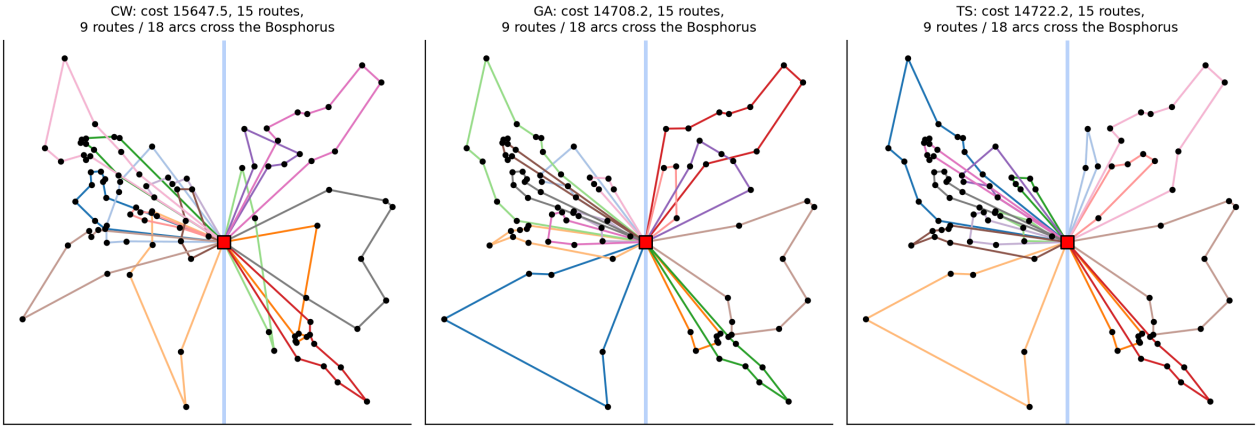


Figure 3: Best plans on our MediDrone instance (depot = red square, Bosphorus = blue line).

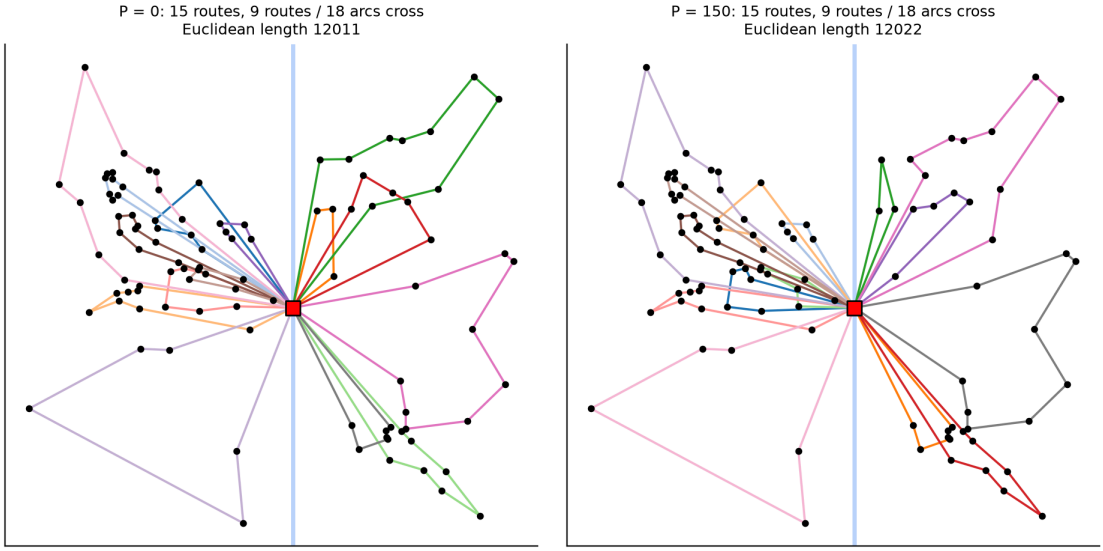


Figure 4: Best of three TS runs (60 s each) on the same customers with $P = 0$ and with $P = 150$.

6 Discussion and Reflection

6.1 Which algorithm is better, and why?

Under equal time budgets the **memetic GA is the more robust method**. It reaches the BKS in 38 of 40 benchmark runs, and its MediDrone result is identical in all ten runs. Tabu Search is equally good on instances with $n \leq 60$ and finds good solutions very early. On A-n80-k10 and MediDrone, however, it stalls 0.1–0.4% above the GA. We see three reasons in our data:

1. **Global vs. local structure.** OX crossover recombines whole route fragments from two good parents, and Split re-optimises where the routes are cut. TS changes at most two routes per iteration, so moving from one good arrangement of the fleet to a different one takes many deteriorating steps. The arc tabu list with tenure 5–15 only partly enables those steps.
2. **Variance.** TS outcomes depend on which basin the trajectory happens to enter (standard deviation 5.3 on A-n80-k10). A population of 25 solutions hedges against this.
3. **The GA contains our best local search.** Figure 5 shows that without local search the GA is 4% above the BKS on A-n60-k9. That is worse than TS, even though it runs 3.7 times more generations. The strength of the GA therefore comes from the **combination** of recombination and an intensive, Tabu-like neighbourhood. This is exactly the hybrid that the literature recommends [15].

TS has its own strengths. It is simpler (one solution, no decoder), it is the fastest to a usable plan (on A-n80-k10 every TS run is within 2% of the BKS after at most 1.3 s, the GA needs up to 2.1 s), and an iteration is cheap enough to scale linearly in n (Section 4). If the CTO needed a plan in under a second, TS would be our choice.

6.2 How did the Bosphorus penalty change the routes?

Less than we expected, and the reason is instructive. Every plan in Figure 3 and both plans in Figure 4 consist of 15 routes: 9 serve only European clinics, 6 serve only Asian clinics, and **none mixes the two sides**. All 9 European routes cross the strait exactly twice (18 crossing arcs), with $P = 150$ and with $P = 0$ alike. Both plans have almost the same Euclidean length: 12 022 vs. 12 011.

The explanation is geometric. The depot lies at $x = 500$, which is formally on the Asian side. Every route that visits Europe must cross at least once on the way out and once on the way back. A route visiting both sides also needs only these two crossings (depot → Europe → Asia → depot). Hence, for **any** plan, the penalty equals $2P$ times the number of routes that visit Europe. With a European demand of 875 and $Q = 100$, at least $\lceil 875/100 \rceil = 9$ such routes are needed, so the penalty is at least $2 \cdot 150 \cdot 9 = 2700$. Our plans attain this bound, as the costs confirm: $14722.2 - 12022.2 = 2700$. In our instance the penalty therefore acts as a **fixed cost per European route**. The only incentive it creates is to pack the European clinics into as few, full routes as possible, and the European routes are indeed loaded to 95–100. It would shape the geometry of the routes much more if the depot were far from the strait.

6.3 Parameter sensitivity

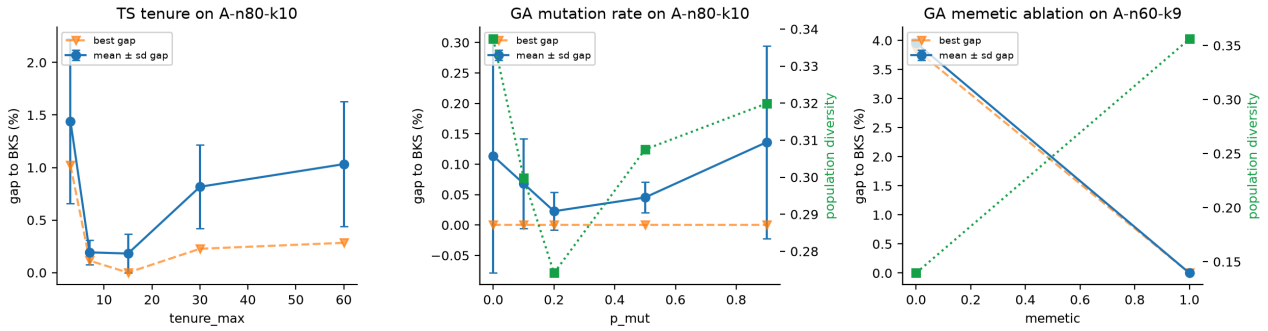


Figure 5: Parameter studies, 5 seeds × 20 s per setting. Left: TS tenure range $[t/3, t]$. Middle: GA mutation probability, with final population diversity (green, mean pairwise share of different arcs). Right: GA with and without local search.

- **Tabu tenure** shows the classic U-shape. With $t = 3$ the search cycles back into the same local optima (mean gap 1.4%). With $t \geq 30$ too many moves are forbidden, and the search is pushed away from good regions (0.8–1.0%). The range 7–15 is best (about 0.2%), which confirms our default $[5, 15]$.
- **Mutation rate** matters much less. Every setting finds the BKS at least once, the mean gap varies only between 0.02% and 0.14%, and the minimum is at our default $p_{\text{mut}} = 0.2$. Final diversity stays at 0.27–0.34 for all rates. The spacing rule and the restarts, not mutation, are what keep the memetic GA diverse.
- **Memetic ablation.** Turning local search off raises the gap from 0% to 4%. At the same time the population diversity collapses from 0.36 to 0.14: without local search, OX + Split converges prematurely to one family of solutions.

6.4 Theory vs. practice

Section 2 proves that we cannot expect a polynomial algorithm with a guarantee of optimality. Approximation theory offers polynomial algorithms with **guaranteed** worst-case ratios. For CVRP with unsplittable demands, the classical iterated tour partitioning bound with Christofides' algorithm is about 3.5. That is a statement about the worst case: a dispatcher would accept 250% extra flight distance. Our metaheuristics give **no** guarantee at all; we could not even bound the number of local-search passes polynomially. Yet in every test where the optimum is known they are within 0–0.25% of it, and they produce a plan in seconds.

Industry accepts algorithms without guarantees for four reasons:

- the worst cases behind approximation ratios rarely look like real instances;
- performance can be **measured** on representative data, which is exactly what CVRPLIB is for;
- the time budget can be traded for quality;
- an independent checker guarantees what really matters operationally: feasibility.

Guarantees still have practical value. Lower bounds such as the radial bound tell us how far a plan **could** be from optimal, and approximation algorithms make safe fallbacks. In practice the two worlds complement each other.

6.5 Personal reflection

What surprised us most. Feasibility, not distance, turned out to be the real difficulty. A-n45-k6 is 98.8% full: Clarke–Wright needs 7 vehicles for $K = 6$, and **none** of the random giant tours we sampled had a Split into 6 capacity-feasible routes. The "hidden Bin Packing problem" from the theory section suddenly became a practical obstacle. Allowing overloaded solutions with an adaptive penalty in both metaheuristics solved it. We were also surprised that the Bosphorus penalty did not change the shape of the routes; the depot placement explains it.

Hardest bug. Our O(1) delta formulas for intra-route relocation and 2-opt* were wrong in rare edge cases: inserting a customer right next to its old position, and segments that touch the depot. These cases do not crash anything. The search simply believes a move is better than it is, so the costs drift slowly. We found it by writing a test that compares every delta formula with a full recomputation on thousands of random moves and on an exhaustive list of edge cases. We also added a `debug_check` mode that recomputes the cost after every applied move. A second subtle issue was floating-point drift on the MediDrone instance after hundreds of thousands of incremental updates, so TS now resynchronises its cost every 1000 iterations.

What we would try next. A hybrid that uses Tabu Search as the GA's local search, adaptive large neighbourhood search, and time windows: vaccines and blood samples have deadlines, which the "deadlines" in our title would finally deserve. An exact branch-and-cut on our MediDrone instance would also tell us whether the GA's plan is truly optimal.

References

[1] P. Toth and D. Vigo, Eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. in MOS-SIAM Series on Optimization. Philadelphia: SIAM, 2014.

[2] G. Clarke and J. W. Wright, "Scheduling of Vehicles from a Central Depot to a Number of Delivery Points," *Operations Research*, vol. 12, no. 4, pp. 568–581, 1964.

[3] C. Prins, "A Simple and Effective Evolutionary Algorithm for the Vehicle Routing Problem," *Computers & Operations Research*, vol. 31, no. 12, pp. 1985–2002, 2004.

[4] F. Glover and M. Laguna, *Tabu Search*. Boston: Kluwer Academic Publishers, 1997.

[5] P. Augerat, J. M. Belenguer, E. Benavent, A. Corberán, D. Naddef, and G. Rinaldi, "Computational Results with a Branch and Cut Code for the Capacitated Vehicle Routing Problem," Research Report 949-M, 1995.

[6] CVRPLIB, "CVRPLIB: Capacitated Vehicle Routing Problem Library." [Online]. Available: http://vrp.galgos.inf.puc-rio.br/

[7] R. M. Karp, "Reducibility among Combinatorial Problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds., New York: Plenum Press, 1972, pp. 85–103.

[8] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979.

[9] C. H. Papadimitriou, "The Euclidean Travelling Salesman Problem is NP-Complete," *Theoretical Computer Science*, vol. 4, no. 3, pp. 237–244, 1977.

[10] M. Haimovich and A. H. G. Rinnooy Kan, "Bounds and Heuristics for Capacitated Routing Problems," *Mathematics of Operations Research*, vol. 10, no. 4, pp. 527–542, 1985.

[11] K. Altinkemer and B. Gavish, "Heuristics for Unequal Weight Delivery Problems with a Fixed Error Guarantee," *Operations Research Letters*, vol. 6, no. 4, pp. 149–158, 1987.

[12] N. Christofides, "Worst-Case Analysis of a New Heuristic for the Travelling Salesman Problem," Management Sciences Research Report 388, 1976.

[13] J. Blauth, V. Traub, and J. Vygen, "Improving the Approximation Ratio for Capacitated Vehicle Routing," *Mathematical Programming*, vol. 197, pp. 451–497, 2023.

[14] A. R. Karlin, N. Klein, and S. Oveis Gharan, "A (Slightly) Improved Approximation Algorithm for Metric TSP," in *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, 2021, pp. 32–45.

[15] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi, and W. Rei, "A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems," *Operations Research*, vol. 60, no. 3, pp. 611–624, 2012.

[16] J.-F. Cordeau, G. Laporte, and A. Mercier, "A Unified Tabu Search Heuristic for Vehicle Routing Problems with Time Windows," *Journal of the Operational Research Society*, vol. 52, no. 8, pp. 928–936, 2001.

[17] P. Toth and D. Vigo, "The Granular Tabu Search and Its Application to the Vehicle-Routing Problem," *INFORMS Journal on Computing*, vol. 15, no. 4, pp. 333–346, 2003.

[18] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

A Contribution table

Member	Main responsibilities	Share
Member 1	Core library: CVRPLIB parser, MediDrone generator, evaluation, independent feasibility checker, exact DP for tiny instances	25%
Member 2	Genetic Algorithm: Split DP, operators, memetic local search; GA sections of the report	25%
Member 3	Clarke–Wright and Tabu Search: moves, delta evaluation, penalty adaptation; TS sections of the report	25%
Member 4	Theory chapter (reductions, Bin Packing, metric proof); experiment scripts, statistics, figures; slides	25%

B Reproducing every table and figure

All commands are run from the repository root after `pip install -e .`; see `README.md` for details.

Command	Produces
`python experiments/make_data.py`	MediDrone instance (seed from `team.json`), tiny instances, exact optima, Fig. of the instance
`python experiments/run_experiments.py`	`results/runs.csv` and one JSON per run (routes and convergence history)
`python experiments/scaling.py`	`results/scaling.json` (Task 6 timings, delta vs. full evaluation)
`python experiments/param_study.py`	`results/param_study.csv` (parameter sensitivity)
`python experiments/bosphorus.py`	`results/bosphorus.json` (P = 0 vs. P = 150)
`python experiments/make_figures.py`	`results/summary.csv`, `results/stats.csv`, all figures
`python report/build.py`	this report as PDF
`pytest`	unit tests (parser, checker, exact DP, Split, GA, CW, TS, delta formulas)

C Sources, own work and tools

All algorithms (parser, generator, Clarke–Wright, Split, GA, Tabu Search, exact DP, checker) are our own implementations. We used no routing or metaheuristic library. Published pseudocode served as a starting point: Clarke–Wright [2], giant tour + Split and the GA design [3], and Tabu Search concepts [4]. Source comments mark where an implementation follows one of these. Benchmark instances and best-known solutions come from CVRPLIB [6]. Standard libraries only were used for numerics (NumPy), plotting (Matplotlib), statistics (SciPy) and typesetting (Typst).